

TUITION MANAGEMENT SYSTEM

Parent and Student Web Portal QA Report

Implementation readiness, requirement traceability, security, and release gates

Assessment date	15 September 2026
Application	React / TypeScript web portal
Assessment status	NOT RELEASE-READY
Classification	Confidential - development use

Release decision

Do not release until all P0 findings and the identified critical Student and Parent workflow findings are closed.

Project: Tuition Management System (TMS)

Application: React/TypeScript web portal

Assessment date: 15 September 2026

Assessment type: Static implementation, requirements, test-coverage, and build-readiness review

Status: Not release-ready

1. Executive Summary

The Parent and Student web portals have broad functional coverage and most required screens are present. Both portals use live account-scoped API data and include responsive layouts, dark-theme styling, loading states, empty states, error recovery, and keyboard-aware payment dialogs.

The web application is not currently deployable because unresolved Git conflict markers in `apps/web/src/pages/TeacherPortal.tsx` cause both the production build and lint command to fail. Although this defect is outside the Parent and Student portal pages, it blocks shipping the entire web bundle.

The most important portal-specific gaps are:

1. Student class-average comparisons are incorrect for published `StudentScore` records because the API returns the student's own score as the class average.
2. The Parent appointment UI supports only Branch Admin requests, despite requirements and backend support for assigned-teacher and group appointments.
3. Alternative-time negotiation and multi-party approval are not implemented in the Parent UI.
4. Portal notifications are reconstructed when data is loaded and read status is kept in browser storage; they are not durable, auditable notification records.
5. Push and SMS behavior is incomplete or backed by mock delivery services.
6. Current portal smoke tests can report success without authenticating when credentials are missing.
7. Large histories are silently limited by API queries without pagination or complete-history controls.

The recommended release sequence is to restore build health, fix academic data correctness, complete appointments, implement durable notification delivery, and then expand authorization and end-to-end regression coverage.

2. Scope and Sources

2.1 In scope

- Student web routes under `/student/*`
- Parent web routes under `/parent/*`
- Portal-specific frontend components and service layers
- Parent and Student portal API projections
- Related appointment, leave, messaging, attendance, billing, certificate, and notification behavior
- Existing browser smoke tests
- Production build and lint readiness
- Responsive, state, interaction, and basic accessibility implementation

2.2 Out of scope

- Flutter mobile portal behavior except where used as background context
- Full penetration testing
- Production payment-provider certification
- Production SMS or push-provider verification
- Destructive database or live-environment testing
- Visual testing against a running authenticated production-like environment

2.3 Requirements sources

- documents/prd-each-section/08_Student.pdf
- documents/prd-each-section/09_Parent_Guardian.pdf
- apps/web/AGENTS.md
- Existing frontend and backend implementation
- Existing Parent and Student smoke tests

3. Assessment Method

The review traced each requirement through the following layers:

1. Required user outcome in the role specification
2. Portal route and visible UI implementation
3. Frontend-to-API service call
4. Server-side tenant, account, child, or student scoping
5. Persistence and delivery behavior
6. Existing automated test coverage
7. Build, lint, responsive, interaction-state, and accessibility signals

3.1 Severity definitions

Priority	Definition
P0 - Blocker	Prevents deployment, basic access, or safe release of the web application.
P1 - Critical	Produces materially incorrect information, violates a core requirement, or leaves an important workflow incomplete.
P2 - Major	Significant usability, reliability, completeness, or maintainability gap.
P3 - Minor	Lower-risk consistency or polish issue that should be scheduled after core correctness.

4. Build and Release Readiness

QA-BUILD-001 - Production build blocked

- Priority: P0
- Status: Open
- Evidence: `npm run build` fails with TypeScript error TS1185: Merge conflict marker encountered.
- Affected file: `apps/web/src/pages/TeacherPortal.tsx`
- Locations observed: approximately lines 18, 71, 85, 3029, 3144, and 3147
- Impact: No production web bundle can be generated, so Parent and Student portal changes cannot be deployed.
- Required action: Resolve the conflict, review both sides of the merge, then run the complete build and portal regression suite.
- Acceptance criteria:
 - No `<<<<<<`, `=====`, or `>>>>>>` markers remain in application source.
 - `npm run build` exits successfully.
 - Generated production assets load without console or routing errors.

QA-BUILD-002 - Lint pipeline blocked

- Priority: P0
- Status: Open
- Evidence: `npm run lint` fails on the same conflict markers.

- Additional observation: Several unrelated lint warnings remain elsewhere in the web application.
- Required action: Resolve the conflict and establish a warning policy for CI.
- Acceptance criteria: `npm run lint` exits successfully with no errors.

5. Student Portal Assessment

5.1 Requirement traceability

Requirement	Implementation status	Notes
Student dashboard	Implemented	Includes today summary, timetable, homework, attendance, result, certificate, and event metrics.
Merged timetable across enrolled course types	Implemented	Active enrolled classes are merged for daily and weekly views.
Pending homework due soon	Implemented	Pending, due-soon, overdue, and completed filters exist.
Published scores	Implemented with defect	Own scores are account-scoped, but one class-average calculation is incorrect.
Trend and strong/weak subject insights	Implemented	Derived from the student's published result history.
Class-average comparison	Partially implemented	Homework-derived grades calculate an average; direct score records use the student's score as the average.
Digital student ID	Partially implemented	Visual ID exists; verifiable branch-scanning workflow was not found.
Attendance history	Implemented with limit	Teacher-marked statuses include present, absent, and excused; only the latest records are returned.
Explicit blocked status and amount owed	Implemented	Blocked banner and outstanding balance are exposed.
Payment calendar and itemized invoice	Implemented	Upcoming, due-soon, overdue, paid, discount, fine, and net-payable states are represented.
Nepal Pay/payment flow	Implemented, provider QA pending	Payment checkout exists; provider certification was outside this audit.
Permanent certificate history	Implemented with query dependence	Available certificates are shown and downloadable; full history depends on backend query behavior.
Full academic calendar and upcoming events	Implemented	Shared calendar component is used.
Student notifications	Partially implemented	In-portal feed exists; persistence and real push delivery are incomplete.
Student portal is read-mostly	Implemented	Academic records and assignments are presented as read-only.

5.2 Open Student findings

QA-STU-001 - Incorrect class-average comparison

- Priority: P1
- Status: Open
- Component: Results and insights
- Evidence: In services/api/src/routes/users.ts, direct studentScore rows map classAverage to Number(row.score).
- Impact: The UI can state that a result is above or below the class average using a value that is actually the student's own score. This is an academic data-correctness defect.
- Required action: Calculate the average from all eligible published scores associated with the same assessment, class, and maximum mark.
- Acceptance criteria:
 - The API returns an independently calculated class average.
 - Only aggregated class data is exposed; no other student's individual result is returned.
- Tests cover above-average, below-average, equal-average, missing-peer, and mixed-maximum cases.

QA-STU-002 - Notification records are not durable

- Priority: P1
- Status: Open
- Component: Notifications
- Evidence: Notifications are constructed from current invoice, homework, attendance, leave, and certificate records during the portal request. Read IDs are saved to `localStorage` by the frontend.
- Impact: Read state does not follow the student between browsers or devices. Notification delivery and history cannot be reliably audited.
- Required action: Introduce persisted notification events and per-user delivery/read records.
- Acceptance criteria:
 - Notifications have stable database IDs.
 - Read/unread state is stored server-side.
 - Single-read and mark-all-read API operations are available.
 - Read state remains consistent across browsers and devices.

QA-STU-003 - Real web push delivery is not demonstrated

- Priority: P1
- Status: Open
- Component: Notifications
- Impact: The PRD calls for push notifications, but an in-portal notification feed is not equivalent to device/browser push delivery.
- Required action: Add a web-push service worker, subscription management, delivery provider, delivery logging, retry policy, and opt-out behavior.
- Acceptance criteria: A signed-in student receives and can open a browser push notification for every configured trigger.

QA-STU-004 - Histories are silently truncated

- Priority: P2
- Status: Open
- Affected data: Attendance, invoices, results, notifications, and potentially certificates
- Evidence: Server queries use fixed take limits such as 60 attendance records, 12 invoices, and 100 direct scores without portal pagination.
- Impact: Users may interpret partial history as complete history.
- Required action: Add cursor pagination, date filters, record counts, and visible "load more" behavior.
- Acceptance criteria: Users can reach their complete permitted history without silent omission.

QA-STU-005 - "Immediate" results are request-refresh based

- Priority: P2
- Status: Open
- Impact: New results appear only after portal data is fetched again. There is no evident polling, server-sent event, or WebSocket update.
- Required action: Define the acceptable freshness interval and implement refetch-on-focus, polling, SSE, or WebSocket behavior.
- Acceptance criteria: A teacher-published score becomes visible within the agreed service-level interval without manual browser refresh.

QA-STU-006 - Digital ID lacks evident verification workflow

- Priority: P2
- Status: Open pending product confirmation
- Impact: A visual card can be copied or altered and may not satisfy "usable for identification at the branch."

- Required action: If operational verification is required, add a signed QR/barcode and a staff verifier that checks student, tenant, branch, active enrollment, and expiry.
- Acceptance criteria: Staff can scan and validate an ID without exposing unnecessary student data.

QA-STU-007 - Loaded homework details are not fully surfaced

- Priority: P2
- Status: Open
- Evidence: Homework data includes description and content/submission URLs, while the visible list focuses on title, subject, teacher, and due state.
- Required action: Add a read-only assignment-detail view and safe attachment links.
- Acceptance criteria: Students can view every teacher-shared homework instruction and permitted attachment.

6. Parent Portal Assessment

6.1 Requirement traceability

Requirement	Implementation status	Notes
Switch between linked children	Implemented	Selected child is carried in the URL and server-validated.
Keep child data separate	Implemented	Portal API validates the requested student against the signed-in parent's links.
View timetable, attendance, invoices, and calendar per child	Implemented	Data is loaded for one selected child at a time.
Parent-visible remarks only	Implemented	Server filters to <code>parentVisible: true</code> .
Assigned-teacher messaging only	Largely implemented	Assigned teachers are derived from current enrollments; Branch Admin contacts are mixed into the same list.
Separate conversation per child	Implemented	Messages are scoped by student ID and parent account.
Individual teacher/admin appointment	Partially implemented	UI creates only Branch Admin appointments. Backend supports both target types.
Configurable minimum booking window	Implemented	Tenant booking-window hours are checked server-side and shown in the UI.
Group appointment approval	Backend foundation only	API has group fields; Parent UI does not expose the workflow.
Alternative-time negotiation	Read-only/partial	Alternative data can be returned, but Parent actions are absent.
Planned leave request	Implemented	Parent can submit a child-scoped request.
Approved leave becomes excused attendance	Backend workflow present; E2E proof needed	Requires integration coverage across approval and attendance creation.
Emergency departure notification	Partially implemented	Feed item exists; real SMS/push delivery remains incomplete/mock-based.
Per-child fees and QR payment	Implemented	Includes itemized breakdown and checkout dialog.
Child certificate download	Implemented	Issued documents are listed per child.
Push and SMS notification policy	Partially implemented	Channel labels exist, but delivery persistence/provider behavior is incomplete.

6.2 Open Parent findings

QA-PAR-001 - Assigned-teacher appointment booking missing from UI

- Priority: P1
- Status: Open
- Evidence: `requestAppointment` is called with `target: 'BRANCH_ADMIN'`. The backend accepts `TEACHER` and `BRANCH_ADMIN`.
- Impact: Parents cannot book required individual meetings with teachers assigned to their child.
- Required action: Add an appointment-target selector populated exclusively from valid contacts for the selected child.
- Acceptance criteria:
 - Parent can choose an assigned teacher or Branch Admin.
 - Unassigned teachers never appear and are rejected if submitted manually.
 - Appointment threads remain child-scoped.

QA-PAR-002 - Group meeting workflow absent from Parent UI

- Priority: P1
- Status: Open
- Impact: Parents cannot request or monitor group appointments as defined by the PRD.
- Required action: Add group-meeting creation, participant selection constrained by authorization, and participant approval status.
- Acceptance criteria: A group appointment is confirmed only after all required participants approve.

QA-PAR-003 - Alternative-time negotiation is incomplete

- Priority: P1
- Status: Open
- Impact: A proposed alternative may be displayed, but the parent cannot accept, reject, or propose another time. The original and proposed times are not presented as a complete linked negotiation timeline.
- Required action: Add explicit negotiation actions and immutable request history.
- Acceptance criteria:
 - Original requested time remains visible.
 - Every alternative is recorded as a linked event.
 - Parent can accept, reject, or counter-propose.
 - Final confirmation identifies the agreed time and participants.

QA-PAR-004 - Production push/SMS delivery incomplete

- Priority: P1
- Status: Open
- Evidence: Leave workflows reference mock push and SMS services; portal data labels expected channels without demonstrating durable delivery.
- Impact: Emergency departure and high-urgency alerts may not reach parents outside the portal.
- Required action: Integrate production providers with templates, consent, retry, delivery status, and failure escalation.
- Acceptance criteria: Delivery can be audited by notification, recipient, channel, attempt, provider result, and timestamp.

QA-PAR-005 - Contact taxonomy is misleading

- Priority: P2
- Status: Open
- Evidence: Branch Admin users are included in the same teachers collection displayed as "Assigned teachers."
- Impact: The UI's privacy explanation does not precisely match the available contacts.
- Required action: Separate "Assigned teachers" and "Branch support" contacts.
- Acceptance criteria: Every contact is clearly labelled by role and remains authorized for the selected child and branch.

QA-PAR-006 - Message delivery and history need dedicated contracts

- Priority: P2
- Status: Open
- Impact: All relevant messages are embedded in the portal payload; no pagination, unread state, delivery state, or realtime update behavior is evident.
- Required action: Add paginated child-and-contact thread endpoints, message status, and defined refresh/realtime behavior.
- Acceptance criteria: Long conversations load incrementally and new messages appear within the agreed freshness interval.

QA-PAR-007 - Parent performance comparison incomplete

- Priority: P2
- Status: Open
- Impact: Parent performance signals show student trends but not independently calculated class comparisons.

- Required action: Reuse the corrected aggregate result service for Parent performance views when authorized by tenant policy.
- Acceptance criteria: Parent sees only approved aggregates and no peer-identifying data.

QA-PAR-008 - Parent histories are silently limited

- Priority: P2
- Status: Open
- Affected data: Attendance, invoices, leave, appointments, certificates, messages, and notifications
- Required action: Add pagination and date/status filters.
- Acceptance criteria: The interface clearly indicates partial results and provides access to all permitted records.

QA-PAR-009 - Roll number is derived from internal student ID

- Priority: P3
- Status: Open
- Impact: A database-ID prefix may not match the institution's official roll or admission number.
- Required action: Use the canonical institution-issued identifier.
- Acceptance criteria: Parent, Student, invoice, certificate, and staff views display the same official identifier.

7. Security and Privacy Assessment

7.1 Controls found

- Student portal lookup is tied to the authenticated user's student record and tenant.
- Parent portal lookup is tied to the authenticated user's parent record and tenant.
- A requested child ID must exist in the parent's studentParents links.
- Parent-visible remarks are filtered server-side.
- Parent messages are filtered by tenant, student, sender, and receiver relationship.
- Teacher contacts are derived from current child enrollments.
- Calendar data uses role-aware audience filtering.

7.2 Required security tests

The following must be tested at the HTTP/integration level rather than only through browser visibility:

1. Student A cannot retrieve Student B's portal projection, certificate, invoice, media, or payment QR.
2. Parent A cannot request a child linked only to Parent B.
3. Parent cannot send a message to an unassigned teacher by changing the receiver ID.
4. Parent cannot book an unassigned teacher or unauthorized branch administrator.
5. One sibling's message thread cannot appear in another sibling's portal context.
6. Revoked certificates cannot be downloaded through a previously known URL.
7. Tenant IDs cannot be overridden using URL, body, query, or forged token data.
8. Payment checkout verifies payer access to the invoice before returning provider details.

8. Automated Test Coverage Assessment

QA-TEST-001 - Smoke tests can pass without portal execution

- Priority: P0
- Status: Open
- Evidence: When environment credentials are missing, Parent and Student tests only assert that the login field is empty and then return.
- Impact: CI may report a passing portal test without login, authorization, portal rendering, or workflow execution.

- Required action: Mark the test as skipped with a visible reason or fail CI environments where portal tests are required.
- Acceptance criteria: A reported pass means the authenticated portal scenarios actually ran.

QA-TEST-002 - Portal tests are not registered as package scripts

- Priority: P1
- Status: Open
- Evidence: apps/web/package.json contains account-related TestCafe scripts but no Parent or Student portal scripts.
- Required action: Add repeatable scripts and CI jobs with seeded credentials/data.

8.1 Minimum Student regression suite

- Authenticated account-scoping and direct-route access
- Today and weekly timetable across every supported course type
- Homework pending, due-soon, overdue, completed, empty, and attachment states
- Correct student score and aggregate class average
- Strongest/weakest subject and trend calculations
- Present, absent, and excused attendance
- Blocked state with exact outstanding amount
- All invoice states and itemized totals
- Payment success, cancellation, failure, duplicate callback, and delayed verification
- Digital ID verification, if adopted
- Certificate authorization, download, unavailable, and revoked states
- Calendar audience rules
- Persistent notification read state
- Keyboard navigation and mobile/tablet/desktop layouts

8.2 Minimum Parent regression suite

- Multiple-child switching and independent datasets
- Rejection of unlinked child IDs
- Parent-visible versus internal remarks
- Assigned-teacher contact filtering
- Separate sibling conversation histories
- Teacher and Branch Admin appointment requests
- Configured booking-window boundary in Nepal time
- Group participant approvals
- Alternative-time acceptance, rejection, and counter-proposal
- Planned leave request, approval, rejection, and excused-attendance creation
- Emergency-departure push/SMS delivery record
- Per-child invoice and payment scoping
- Per-child certificate authorization
- Persistent per-child notifications
- Keyboard, modal focus, responsive, and dark-theme behavior

9. UX, Accessibility, and Responsive Review

9.1 Positive implementation signals

- Real button, a, input, select, and textarea elements are used for major interactions.

- Portal CSS provides visible :focus-visible styles.
- Payment dialogs include Escape handling, focus containment, and focus restoration in key flows.
- Loading skeletons, empty states, and recoverable load errors are present.
- Dark-theme rules exist for both portals.
- Responsive breakpoints exist for desktop, tablet, and small-screen layouts.
- prefers-reduced-motion handling exists in both portal stylesheets.
- Important payment QR images include dimensions and descriptive alternative text.

9.2 Remaining UX QA

- Run authenticated visual regression at 375 px, 768 px, and 1280 px for every portal view.
- Test 200% browser zoom and large text.
- Verify touch targets are at least 40 by 40 CSS pixels.
- Run automated accessibility scanning and manual keyboard/screen-reader checks.
- Verify dialogs return focus to the exact triggering invoice action, including history-list actions.
- Confirm all currency values use consistent formatting and accessible labels.
- Confirm status is not conveyed by colour alone.
- Verify table behavior and headers on narrow screens.
- Confirm long names, course titles, branch names, and translated copy do not overflow.

10. Recommended Implementation Plan

Phase 1 - Restore release health

1. Resolve the TeacherPortal.tsx merge conflict.
2. Run build and lint until both pass.
3. Add Parent and Student portal test scripts.
4. Make missing test credentials an explicit skip or CI failure, never a false pass.

Phase 2 - Correct critical data and workflows

1. Correct the student class-average calculation.
2. Add API integration tests for result aggregation and privacy.
3. Expose assigned-teacher appointment booking.
4. Implement alternative-time actions and immutable negotiation history.
5. Complete group appointment approval and final confirmation.

Phase 3 - Make notifications production-ready

1. Create persisted notification-event and delivery/read models.
2. Add per-account read APIs.
3. Integrate production SMS and web-push providers.
4. Add delivery logging, retries, dead-letter handling, preferences, and consent.
5. Verify tenant-level notification policy behavior.

Phase 4 - Complete history and freshness

1. Add pagination and filters to all capped datasets.
2. Add refetch-on-focus and a defined live-update mechanism.
3. Add message thread pagination and delivery state.
4. Decide and implement the operational digital-ID verification model.

Phase 5 - Release validation

1. Run authorization integration tests.
2. Run seeded Parent and Student end-to-end tests.
3. Complete responsive and accessibility review.
4. Complete payment-provider and notification-provider UAT.
5. Obtain product-owner sign-off against the traceability tables.

11. Release Gate Checklist

The Parent and Student web portals should not be marked release-ready until every mandatory item below passes:

- [] Production build succeeds.
- [] Lint completes without errors.
- [] No unresolved Git conflict markers remain.
- [] Class-average comparisons are mathematically and privacy-correct.
- [] Parent can book assigned teachers and Branch Admins.
- [] Alternative-time negotiation is actionable and auditable.
- [] Group meetings require all configured approvals.

- [] Student and Parent notification state is persisted server-side.
- [] Required push and SMS alerts use production delivery services.
- [] Portal E2E tests authenticate and execute real scenarios.
- [] Cross-student, cross-child, cross-branch, and cross-tenant access tests pass.
- [] Payment UAT covers success and failure paths.
- [] Full-history access is available or limits are clearly disclosed.
- [] Accessibility and responsive checks pass at agreed target sizes.
- [] Product owner approves any deferred P2 or P3 item in writing.

12. Final Assessment

The portals are beyond the prototype stage and already cover most day-to-day Parent and Student needs. The remaining work is concentrated in release safety, data correctness, workflow depth, delivery infrastructure, and regression proof.

Current recommendation: Do not release the web application until all P0 findings and QA-STU-001, QA-PAR-001, QA-PAR-002, QA-PAR-003, and QA-PAR-004 are closed. P2 findings may be scheduled after release only with explicit product-owner acceptance and clear disclosure of history or freshness limitations.